CLOUDOPS
ENTERPRISE PLATFORM

Cloud Infrastructure • Containerization • CI/CD • Monitoring

FINAL PROJECT REPORT

Submitted by: Parth Chikane

Submission date: 12 November 2026

GitHub: github.com/Parth-727/CloudOps-Enterprise-Platform

AWS Region: ap-south-1 (Mumbai)

A practical implementation of a web-based cloud operations dashboard with a Spring Boot API, PostgreSQL, Docker, Jenkins CI/CD, AWS infrastructure, local Kubernetes orchestration, and Prometheus/Grafana monitoring.

1. Executive Summary

CloudOps Enterprise Platform is a cloud-operations dashboard intended to provide a consolidated view of infrastructure inventory, deployments, application readiness and monitoring. The implemented stack combines a React/TypeScript frontend served through Nginx, a Java 17 Spring Boot backend, PostgreSQL 16, Docker containers, Jenkins automation, AWS infrastructure managed with Terraform, and a local Kubernetes deployment for orchestration validation.

The application was previously deployed to an AWS EC2 instance in the Mumbai region and was also deployed locally to Docker Desktop Kubernetes. The final verification shared for this report confirms successful backend API responses and three Kubernetes workloads reporting Ready/Running.

2. Objectives

• Build a web dashboard for cloud infrastructure and deployment visibility.

• Separate the frontend, backend API and relational database into deployable components.

• Containerize application services with Docker.

• Automate frontend and backend build stages through Jenkins integrated with GitHub.

• Provision and manage AWS infrastructure using Terraform.

• Validate Kubernetes deployment and service connectivity on a local Docker Desktop cluster.

• Expose backend metrics to Prometheus and visualize available telemetry in Grafana.

3. Technology Stack and Architecture

Layer	Technology / implementation
Frontend	React, TypeScript, Vite; Nginx container
Backend	Java 17, Spring Boot, Maven
Database	PostgreSQL 16
Containerization	Docker and Docker images
CI/CD	Jenkins, GitHub
Cloud / IaC	AWS EC2, S3, IAM; Terraform
Orchestration	Kubernetes on local Docker Desktop
Monitoring	Prometheus and Grafana

Request path: Browser → Nginx frontend → /api reverse proxy → Spring Boot backend → PostgreSQL. Prometheus scrapes the backend metrics endpoint; Grafana visualizes selected metrics. Docker images are used by the Jenkins build and Kubernetes deployment.

4. Deployment and Infrastructure

AWS deployment: The application is reported live at http://3.109.48.131 on EC2 instance i-086156740a4667e99 in ap-south-1. The existing AWS deployment and Terraform-managed infrastructure were completed earlier. This submission process does not modify the AWS environment.

Local Kubernetes: Kubernetes context is docker-desktop, with node desktop-control-plane. The manifest is kubernetes/cloudops.yaml. The frontend, backend and PostgreSQL pods were each reported as 1/1 Running. The frontend service uses NodePort 30081, backend service NodePort 30080, and PostgreSQL service port 5432. Since

direct NodePort access on Docker Desktop was unsuccessful, the frontend was accessed using kubectl port-forward service/cloudops-frontend-service 8082:80 at http://localhost:8082.

Container artifacts: cloudops-backend:ci and cloudops-frontend:ci. Frontend container configuration is in frontend/Dockerfile and frontend/nginx.conf.

5. CI/CD Pipeline

Jenkins job: CloudOps-Enterprise-Pipeline (http://localhost:8081). The owner reported pipeline #11 as the latest confirmed successful run. Frontend build and backend Maven package stages passed, and the backend Docker image cloudops-backend:ci was built. Automated backend tests were skipped; therefore, a successful package/build must not be interpreted as a passing automated test suite.

The supplied Jenkins screenshot depicts successful build #3, not build #11. It is included as supporting visual evidence of a successful Jenkins run, but does not independently verify build #11.

6. Verification Results

Check	Observed result
Frontend	HTTP 200 OK through Nginx at localhost:8082
GET /api/status	HTTP 200; status UP; Spring Boot backend
GET /api/dashboard/summary	HTTP 200; 3 deployments; 3 active services; 0 failed deployments
Kubernetes pods	Frontend 1/1 Running; backend 1/1 Running; PostgreSQL 1/1 Running
Prometheus	cloudops-backend target shown as 1/1 UP in supplied screenshot
Grafana	HTTP request-rate and CPU panels show data; 95th-percentile duration panel shows No data

The dashboard summary API returned deployments=3, activeServices=3, totalServices=4, systemHealth=Operational, failedDeployments=0 and uptime=99.9%. These are application-reported dashboard values, not an independent measurement of real-world service availability.

7. Limitations and Scope Notes

• Kubernetes validation was performed on local Docker Desktop, not on Amazon EKS or another managed Kubernetes service.

• The backend Maven package stage skipped automated tests.

• Some Grafana panels did not return data; in particular, the supplied screenshot shows No data for HTTP request duration at the 95th percentile.

• The Prometheus screenshot shows the target UP, but its displayed last-scrape age was approximately 28 minutes. This is point-in-time evidence, not proof of continuous current health.

• The AWS infrastructure and application deployment were already completed; no AWS changes are part of this final packaging activity.

• Big Data/ML and no-code deliverables have not been asserted as completed because the original assignment rubric was not provided for verification.

8. Repository and Submission References

GitHub repository: https://github.com/Parth-727/CloudOps-Enterprise-Platform

Branch: main
Latest reported project commit: e176d7e — Add Kubernetes frontend deployment and container configuration
Key paths: kubernetes/cloudops.yaml; frontend/Dockerfile; frontend/nginx.conf; submission/evidence/

Live AWS application: http://3.109.48.131
Local Kubernetes frontend: http://localhost:8082 (requires the existing port-forward process)
Jenkins: http://localhost:8081
Grafana: http://localhost:3000

9. Conclusion

The project demonstrates an end-to-end cloud-operations application workflow spanning a web frontend, REST backend, database, container builds, CI/CD automation, AWS infrastructure, local Kubernetes orchestration and monitoring integration. The submitted evidence supports successful application/API and local Kubernetes checks, a successful Jenkins build, and a Prometheus target shown UP. The stated limitations are retained to keep the report technically accurate.

Appendix A — Visual Evidence

The following screenshots were supplied for inclusion in the final submission on 12 November 2026.

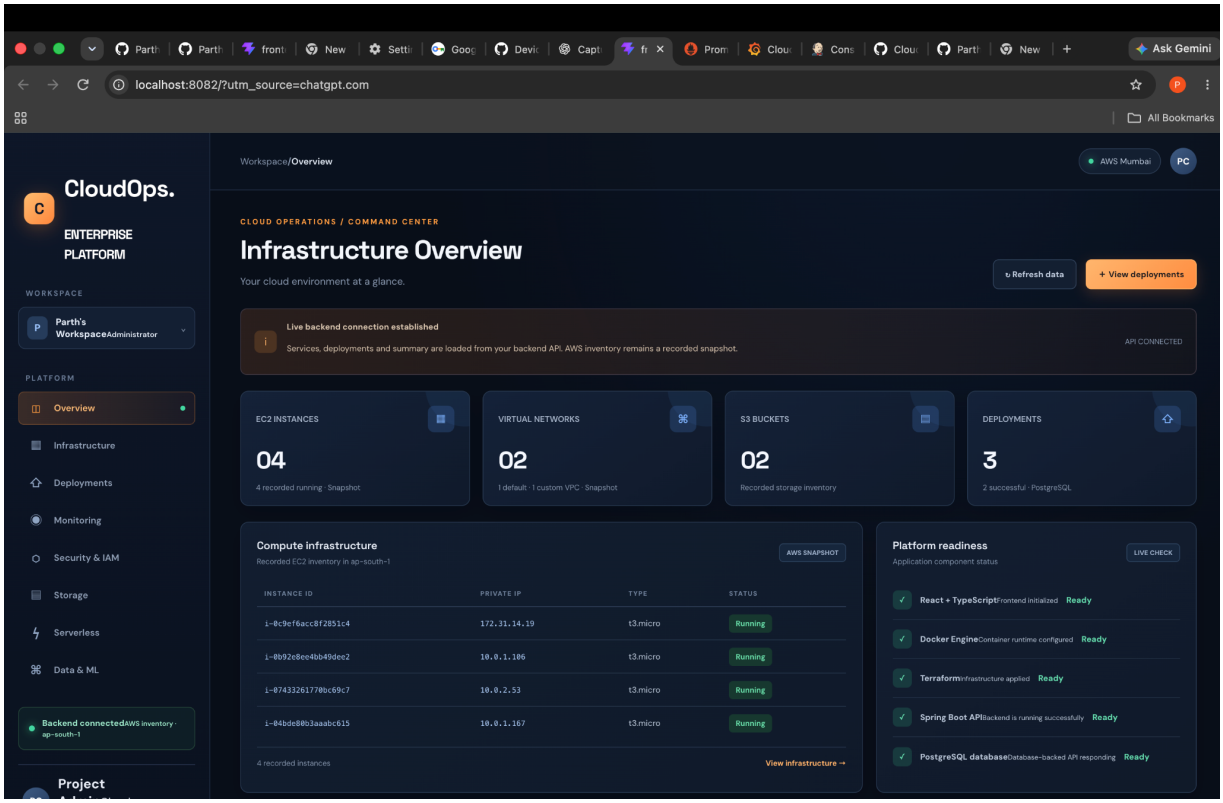


Figure A1. CloudOps application dashboard showing backend connection established and infrastructure overview.

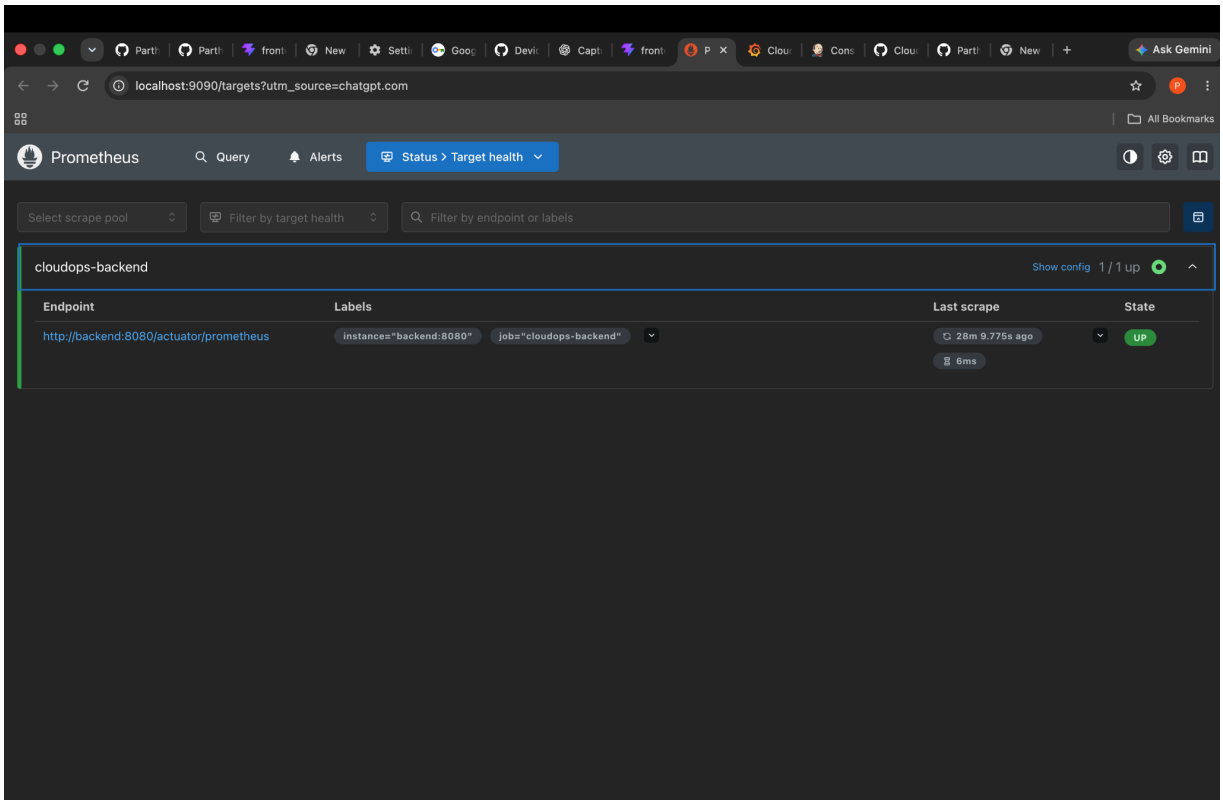


Figure A2. Prometheus target health showing cloudops-backend as 1/1 UP; displayed last-scrape age is approximately 28 minutes.

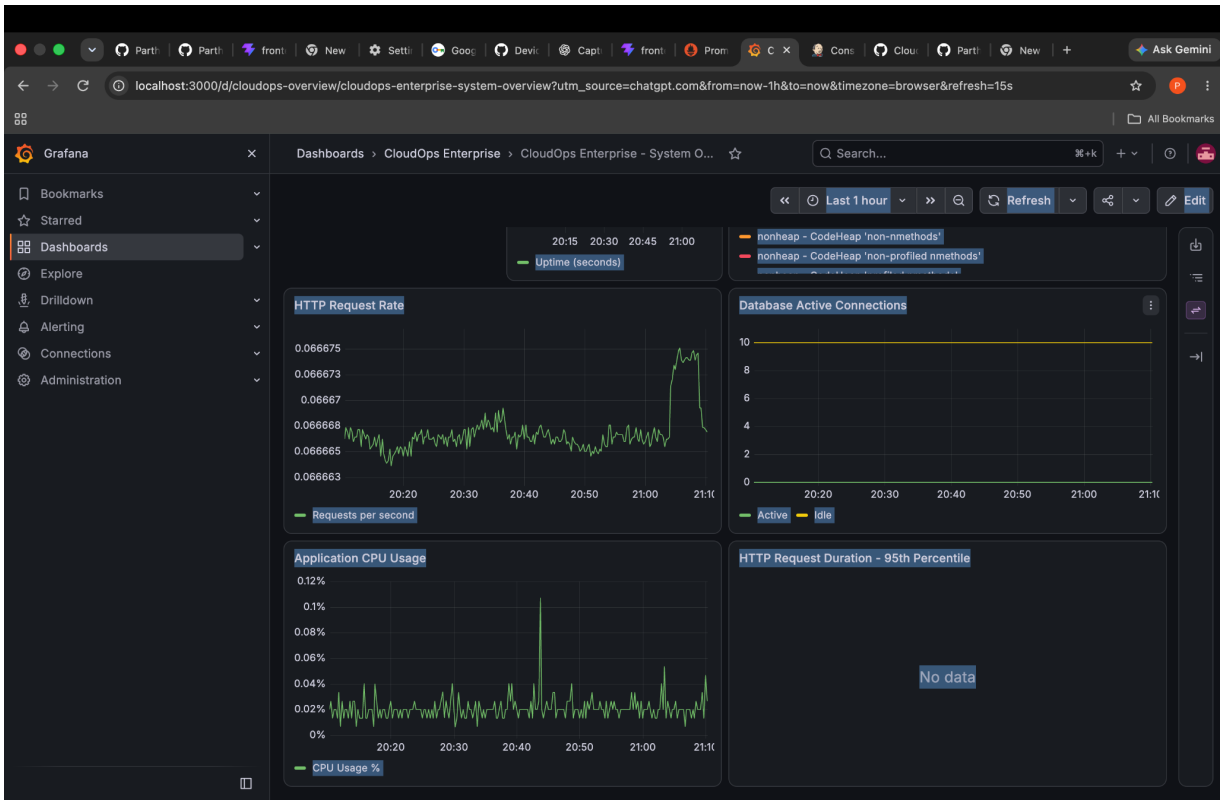


Figure A3. Grafana dashboard; request-rate and CPU panels contain plotted data while HTTP 95th-percentile duration shows No data.

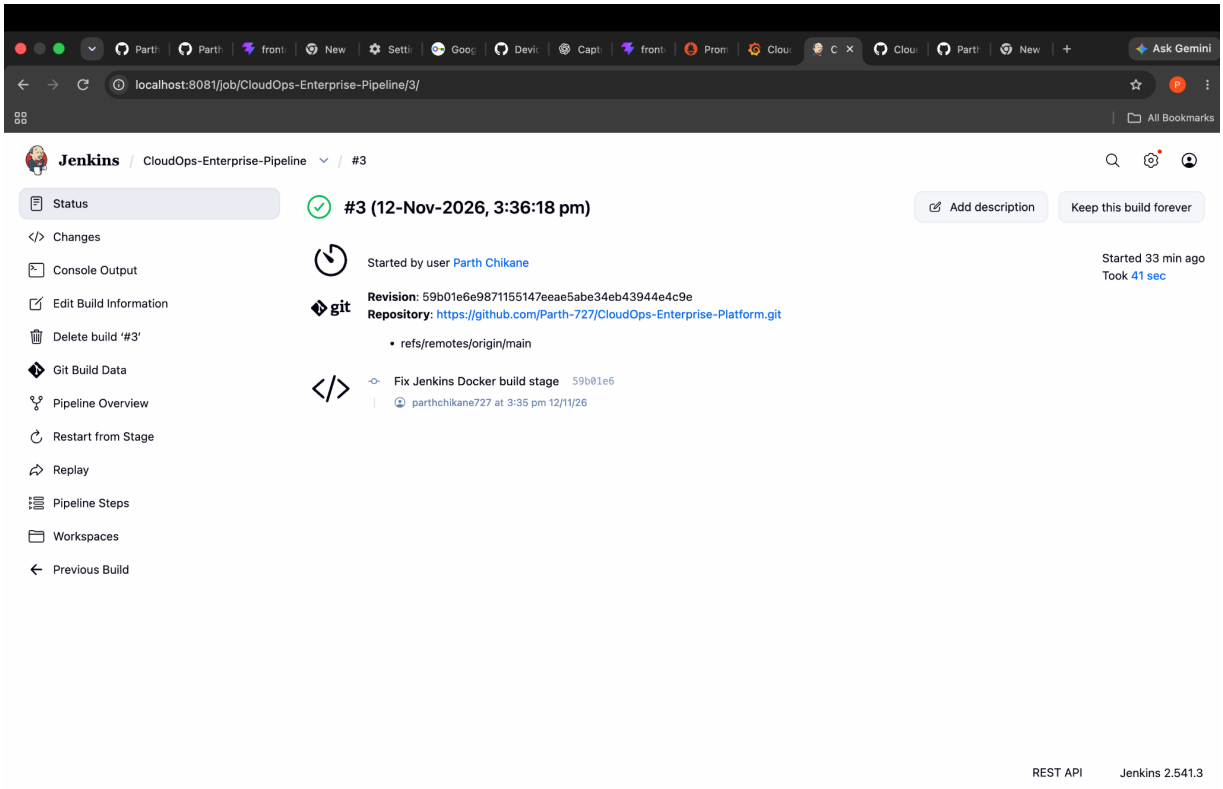


Figure A4. Jenkins successful build #3. The latest successful build #11 is separately reported by the project owner.